

SERVICIO NACIONAL DE APRENDIZAJE – SENA

ACTIVIDAD DE APRENDIZAJE – ADSO

MICROSERVICIOS Y TECNOLOGÍAS PARA EL PROYECTO

1. IDENTIFICACIÓN DE LA ACTIVIDAD

Aspecto	Descripción
Programa	Análisis y Desarrollo de Software – ADSO
Tema	Microservicios y tecnologías para el proyecto
RAP relacionado	Desarrollar la arquitectura de software de acuerdo con el patrón de diseño seleccionado
Tipo	Teórico-práctica – Aprendizaje Basado en Proyectos
Modalidad	Equipos de 3 a 4 aprendices
Duración sugerida	12 horas
Producto	Propuesta de arquitectura basada en microservicios para el proyecto formativo

2. OBJETIVO

Analizar la arquitectura basada en microservicios y seleccionar las tecnologías necesarias para implementar una solución distribuida, teniendo en cuenta requerimientos, atributos de calidad, comunicación, persistencia, seguridad, despliegue y monitoreo.

Al finalizar, el aprendiz estará en capacidad de justificar técnicamente cuándo utilizar microservicios y qué tecnologías son apropiadas para el proyecto.

3. SITUACIÓN PROBLEMA

Una empresa requiere una plataforma para gestionar usuarios, productos, pedidos, pagos y notificaciones. El sistema debe permitir que diferentes módulos evolucionen independientemente y soportar el crecimiento progresivo de usuarios y transacciones.

Inicialmente se propone construir todo como una única aplicación. El reto consiste en diseñar una propuesta basada en microservicios y justificar técnicamente las tecnologías seleccionadas.

4. ACTIVIDAD 1 – COMPRENDIENDO LOS MICROSERVICIOS

1. ¿Qué es una arquitectura de microservicios?
2. ¿Cuál es la diferencia entre un monolito y una arquitectura de microservicios?
3. ¿Qué características debe tener un microservicio?
4. ¿Qué ventajas ofrece?
5. ¿Qué problemas puede generar?
6. ¿Cuándo NO sería recomendable utilizar microservicios?
7. ¿Qué relación existe entre microservicios y escalabilidad?
8. ¿Qué relación existe entre microservicios y DevOps?

Característica	Monolito	Microservicios
Despliegue		
Escalabilidad		
Mantenimiento		

Base de datos		
Comunicación		
Fallos		
Complejidad		
Tecnologías		

5. ACTIVIDAD 2 – IDENTIFICACIÓN DE MICROSERVICIOS

Utilizando el proyecto formativo, identifique los posibles servicios. El siguiente ejemplo sirve como referencia.

Microservicio	Responsabilidad	Datos	API	Dependencias
Usuarios	Gestión de usuarios	usuarios	REST	Auth
Productos	Gestión del catálogo	productos	REST	—
Pedidos	Gestión de pedidos	pedidos	REST	Productos
Pagos	Procesamiento de pagos	pagos	REST	Pedidos
Notificaciones	Envío de notificaciones	notificaciones	REST/Mensajería	Pedidos

Producto: tabla de microservicios reales del proyecto.

6. ACTIVIDAD 3 – SELECCIÓN DE TECNOLOGÍAS

- Backend: Java + Spring Boot, Node.js + Express/NestJS, .NET, Python + FastAPI/Django, PHP + Laravel.
- Comunicación: REST, HTTP/JSON, gRPC, WebSocket y mensajería.
- Mensajería: RabbitMQ, Apache Kafka, MQTT.
- Bases de datos: MySQL, PostgreSQL, MongoDB, Redis.
- Contenedores: Docker, Docker Compose, Kubernetes.
- API Gateway: Spring Cloud Gateway, Kong, NGINX, Traefik.
- Seguridad: JWT, OAuth 2.0, OpenID Connect, Keycloak.
- Monitoreo: Prometheus, Grafana, ELK/Elastic Stack.

Regla: cada tecnología seleccionada debe estar relacionada con una necesidad o requerimiento del proyecto.

7. ACTIVIDAD 4 – MATRIZ TECNOLÓGICA

Necesidad	Tecnología propuesta	Alternativa	Justificación
Backend			
API			
Mensajería			
BD transaccional			
Cache			
Contenedores			
Gateway			
Seguridad			
Monitoreo			

8. ACTIVIDAD 5 – ARQUITECTURA PROPUESTA

Diseñe el diagrama de arquitectura del proyecto. Debe mostrar cliente, API Gateway, microservicios, bases de datos, comunicación, servicios externos, seguridad y tecnologías seleccionadas.

Modelo de referencia: CLIENTE → API GATEWAY → USUARIOS / PRODUCTOS / PEDIDOS → PAGOS / NOTIFICACIONES.

9. ACTIVIDAD 6 – COMUNICACIÓN ENTRE MICROSERVICIOS

Compare comunicación síncrona y asíncrona. Ejemplo síncrono: Pedido → HTTP/REST → Productos. Ejemplo asíncrono: Pedido → RabbitMQ/Kafka → Notificaciones.

1. ¿Dónde utilizaría REST?
2. ¿Dónde utilizaría mensajería?
3. ¿Qué eventos produciría el sistema?
4. ¿Qué eventos consumiría cada servicio?
5. ¿Qué ocurre si un consumidor está temporalmente fuera de servicio?

10. ACTIVIDAD 7 – BASE DE DATOS POR MICROSERVICIO

Analice el principio: cada microservicio debe ser dueño de los datos que necesita para cumplir su responsabilidad.

Ejemplo: Usuarios → BD Usuarios; Productos → BD Productos; Pedidos → BD Pedidos; Pagos → BD Pagos.

1. Analice ventajas y desventajas.
2. Explique problemas de consistencia.
3. Analice transacciones distribuidas.
4. Explique cómo se mantiene la integridad.
5. ¿Qué problemas aparecerían si todos acceden directamente a una única base de datos?

11. ACTIVIDAD 8 – SEGURIDAD

Diseñe autenticación y autorización. Modelo: Usuario → Autenticación → JWT → API Gateway → Microservicios.

- ¿Dónde se autentica el usuario?
- ¿Quién genera el token?
- ¿Cómo se valida el JWT?
- ¿Qué roles existirán?
- ¿Cómo se protegen las APIs?
- ¿Cómo se manejan tokens expirados?
- ¿Cómo se protegen las comunicaciones?

12. ACTIVIDAD 9 – DOCKERIZACIÓN

Elabore una propuesta de contenedores. Ejemplo: api-gateway, usuarios, productos, pedidos, pagos, notificaciones, PostgreSQL/MySQL, Redis y RabbitMQ.

Entregable: archivo docker-compose.yml. Opcionalmente, implemente al menos dos microservicios funcionales y demuestre su comunicación.

13. ACTIVIDAD 10 – ANÁLISIS DE FALLOS

- Escenario A: el microservicio de pagos deja de funcionar. Determine qué operaciones pueden continuar y cómo se informa o reintenta el fallo.
- Escenario B: el servicio de notificaciones está caído. Determine si debe fallar el pedido y cómo una cola puede evitarlo.
- Escenario C: las consultas de productos aumentan diez veces. Determine qué servicio debería escalar y cómo Redis podría ayudar.

14. PRODUCTO FINAL

1. Introducción y descripción del proyecto.
2. Justificación de la utilización de microservicios.
3. Microservicios identificados y responsabilidades.
4. Diagrama general de arquitectura.
5. Matriz de tecnologías con justificación.
6. Comunicación: REST, gRPC, mensajería y eventos.
7. Persistencia y base de datos por servicio.
8. Seguridad: autenticación y autorización.
9. Contenedores y estrategia Docker.
10. Diagrama de despliegue.
11. Manejo de fallos y resiliencia.
12. Escalabilidad.
13. Conclusiones.

15. EVIDENCIAS DE APRENDIZAJE

Evidencia	Producto
1	Matriz monolito vs. microservicios
2	Identificación de microservicios
3	Matriz de selección tecnológica
4	Diagrama de arquitectura
5	Diagrama de comunicación
6	Arquitectura de persistencia
7	Diseño de seguridad
8	docker-compose.yml
9	Documento de arquitectura
10	Sustentación técnica

16. RÚBRICA DE EVALUACIÓN

Criterio	Excelente	Bueno	Básico	Por mejorar
Identificación de microservicios	20%	16%	12%	8%
Selección de tecnologías	20%	16%	12%	8%
Diseño arquitectónico	20%	16%	12%	8%

Comunicación e integración	15%	12%	9%	6%
Seguridad	10%	8%	6%	4%
Persistencia	5%	4%	3%	2%
Docker/despliegue	5%	4%	3%	2%
Sustentación	5%	4%	3%	2%
TOTAL	100%			

17. RETO ADICIONAL

Construya dos microservicios reales que se comuniquen entre sí. Demuestre dos proyectos independientes, dos APIs, comunicación, persistencia, manejo de errores, Docker, documentación de APIs y pruebas mediante Postman.

Resultado esperado: el aprendiz debe poder responder qué microservicios necesita su proyecto, qué tecnología utilizará, cómo se comunicarán, dónde almacenarán sus datos, cómo estarán protegidos y cómo serán desplegados.

18. OBSERVACIONES DEL INSTRUCTOR
